

ZeroBug

AI Agent That Reads Bug Reports, Writes the Fix, and Ships It

Hackathon Submission • Agentic Workflows with Guardrails • DevSecOps

We are building an autonomous AI agent that ingests bug reports from Jira, GitHub issues, SAST, DAST, OSS, and Rails Guard — analyses the root cause, writes the fix, opens a PR, deploys to a test environment for verification, and upon human approval, merges and deploys to production.

1 Idea Title

ZeroBug — AI Agent That Reads Bug Reports, Writes the Fix, and Ships It

2 One-Line Pitch

We are building an autonomous AI agent that ingests bug reports from Jira, GitHub issues, SAST, DAST, OSS, and Rails Guard — analyses the root cause, writes the fix, opens a PR, deploys the fix to a test environment for verification, and upon human approval, merges and deploys to production automatically.

Report in

Root cause

Fix + PR

Deploy to test env

Human approval

Merge + prod deploy

3 Problem Statement

Software engineering teams are trapped in a remediation loop that never ends. Every day, bug reports pile up from multiple directions — QA engineers raise tickets in Jira, developers open GitHub issues, and automated scanners generate findings from SAST, DAST, OSS dependency audits, and Rails Guard policy checks. Each report is detailed, actionable, and waiting. Yet the fix rarely comes fast, because everything that happens after the report lands is entirely manual.

A developer must read the report, understand the context, navigate the codebase to find the right file and function, reason about the root cause, write a correct fix, open a pull request, and hand it off for review. Meanwhile the next report has already arrived. The backlog grows faster than any team can drain it.

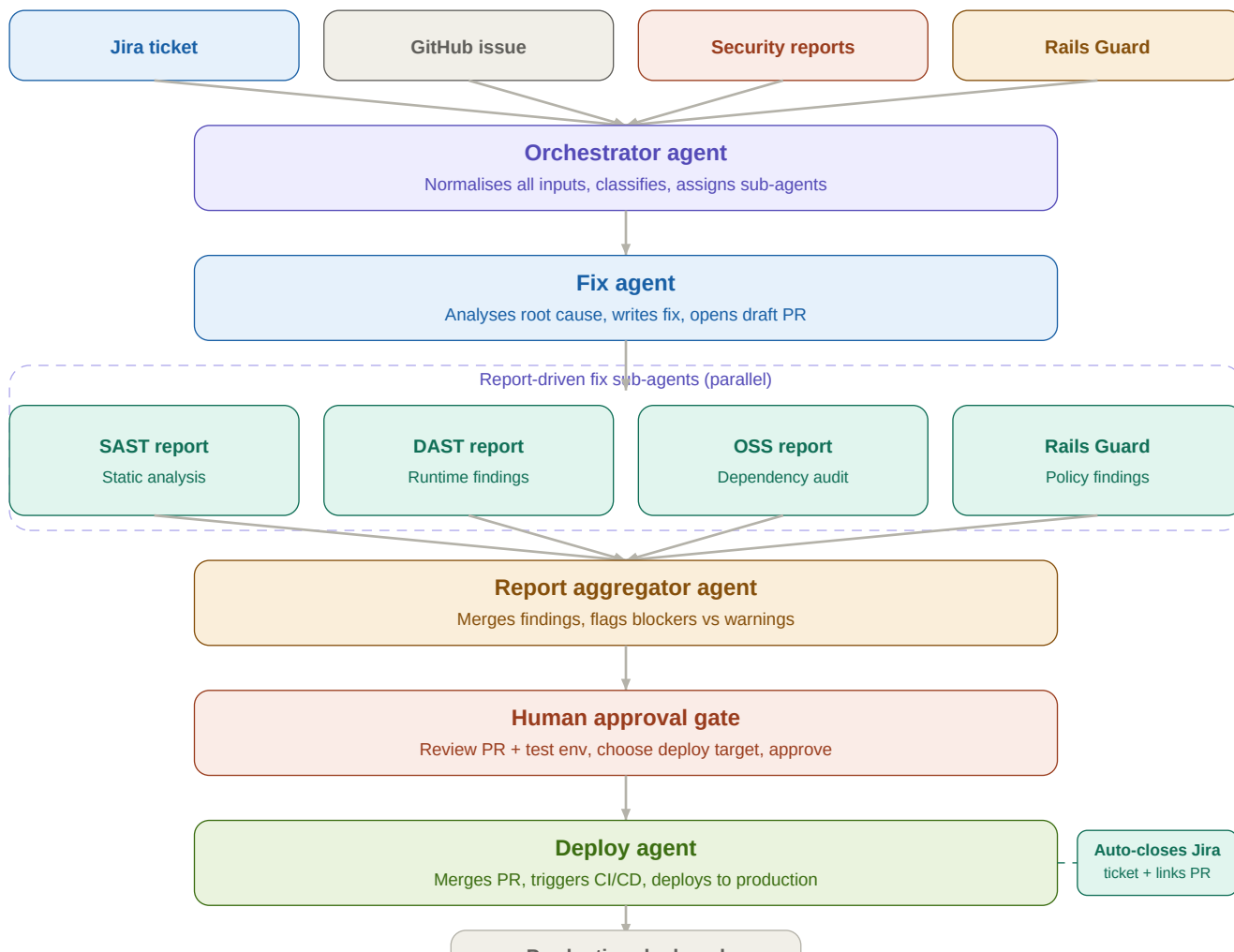
The cost is significant. Developers spend up to 33% of their working time on bug fixing and remediation — time taken directly away from building product. Security backlogs at mid-to-large engineering teams commonly hold hundreds of open findings, with mean-time-to-remediate stretching into weeks for lower-severity issues. The 2023 Verizon DBIR found that 60% of breaches involved a known, unpatched vulnerability — not because teams were negligent, but because the queue was too long and the process too slow.

The critical insight is this: most teams already have the scanners. SAST, DAST, OSS auditors, and Rails Guard are running and producing accurate, structured reports. The problem is not detection — it is the dead zone between detection and fix. No tool today reads a report, identifies the responsible code, generates a correct patch, deploys it to a test environment for verification, and moves it through to production with a human sign-off.

ZeroBug is built to eliminate that dead zone entirely — turning every bug report, from every source, into a verified, human-approved production fix without a developer having to touch it.

★ Agent Pipeline Diagram

ZeroBug — Agent Pipeline



4 AI Tools to Be Used

Claude (Anthropic) — Core reasoning engine. Reads and interprets natural language bug reports from Jira and GitHub, understands structured security findings from SAST, DAST, OSS, and Rails Guard reports, identifies root cause in the codebase, and generates a correct, context-aware fix. Claude's long context window allows reasoning across large files and multi-file dependencies.

Claude tool use / multi-agent orchestration — The orchestrator agent uses Claude's tool use capability to coordinate specialised sub-agents: a report parser, root-cause analyser, fix generator, PR agent, test-env deploy agent, and Jira status updater. Each agent calls external APIs (Jira REST, GitHub API, CI/CD webhooks) as tools.

Vector embeddings + semantic code search — Used to locate relevant source files and functions given a natural language or structured bug description. Embeds the finding and retrieves semantically similar code chunks, dramatically reducing the search space before Claude reasons over results.

LLM-based report normaliser — Converts heterogeneous inputs — free-text Jira tickets, JSON SAST outputs, DAST HTML reports, OSS audit manifests — into a unified finding schema before the orchestrator processes

them.

5 Proposed AI Solution

Step 1 — Ingest and normalise: ZeroBug connects to Jira, GitHub, and security report outputs. An LLM-based normaliser agent reads each input and converts it into a unified finding schema: affected component, severity, description, and reproduction context.

Step 2 — Root cause analysis: The orchestrator agent, powered by Claude, uses semantic code search to retrieve the most relevant files and functions from the codebase. It reasons over the code in context — understanding not just what line is flagged, but why it is wrong and what a correct fix looks like.

Step 3 — Fix generation and PR: The fix agent writes a targeted code change scoped precisely to the root cause. It opens a pull request on the dev branch with a full explanation: what was found, what was changed, and why — human-readable in 60 seconds.

Step 4 — Test environment deployment: The deploy agent spins up the current dev branch — including the fix — on an isolated test environment. A real deployment, not a simulation. The test environment URL is posted to the PR and the originating Jira ticket.

Step 5 — Human approval gate: The engineer receives: the diff, the agent's reasoning, and the test environment link. They approve, request changes, or reject. This is the only human step — and a hard architectural requirement. No merge, no production deploy without explicit human decision.

Step 6 — Merge and production deploy: On approval, the agent merges the PR, triggers the production deployment pipeline, monitors the rollout, and closes the Jira ticket automatically with a link to the merged PR and confirmation of deployment.

6 Expected Benefits

From report to production fix in under an hour — The full journey happens autonomously. A QA engineer raises a Jira ticket at 6pm, goes home, and returns to find the ticket closed, the PR merged, and the fix live in production.

Works on top of the tools your team already uses — ZeroBug does not replace any scanner or issue tracker. It plugs in downstream of Jira, GitHub, SAST, DAST, OSS auditors, and Rails Guard — consuming their outputs as inputs. Zero migration, zero disruption.

Production is never touched without a human saying yes — The agent deploys to a test environment first. The engineer sees the diff, the agent's reasoning, and the test result — then approves with one click. Speed is not traded for safety.

Engineering capacity freed for work that matters — Automating the repetitive 30–40% of developer time returns senior engineering focus to architecture and product innovation. For a team of ten, that is three to four engineers freed from reactive work every week.

Every fix is self-documenting — The agent logs its reasoning — which report triggered the fix, which lines were changed, what the test environment result was — directly to the PR and Jira ticket. Full audit trail, zero documentation overhead.

Scales with the backlog, not with headcount — A startup team of five can maintain the security and quality posture of an enterprise team of fifty. Remediation capacity becomes a function of compute, not hiring.

7 Primary Theme & Industry Domain

Primary Theme	Agentic workflows with guardrails
Secondary Theme	Context graphs & decision augmentation
Industry / Domain	Software / Tech • DevSecOps • Cybersecurity